Deep Learning

Jerónimo Arenas García
Data Processing
Master in Telecommunication Engineering

Universidad Carlos III de Madrid

Outline

1. **Neural networks from scratch**
   Everything with numpy: topology, loss, forward, backward, gradient descent.
   `DL1_NeuralNetworks_numpy.ipynb`
2. **Deep networks with PyTorch**
   The same models with a library that computes the gradients, plus optimization and generalization.
   `DL2_DeepNetworks_PyTorch.ipynb`
3. **Convolutional networks**
   A topology designed for images, data augmentation and transfer learning.
   `DL3_ConvolutionalNetworks.ipynb`

How these sessions work

- Short blocks of theory, then you work on the notebook.
- All notebooks run in Google Colab with GPU.
- Local deployments need to install PyTorch.

Session 1 • notebook DL1

Neural networks from scratch

Why now?

Feed-forward neural networks have been around since the 1960s, but only in the last 10–15 years have they met their expectations.

- Computation at scale (GPUs, cloud)
- Very large datasets
- Topologies suited to particular problems (CNNs, RNNs, Transformers)
- Training strategies that generalize better

You already know SLPs

We start from something you already know: logistic regression.
A single-layer network with logistic activation and cross-entropy loss *is* logistic regression.

The two datasets

Sign language digits

- 2062 images, 64×64, grayscale
- 10 classes, one-hot encoded
- Also used as a binary problem (digits 0 vs 1)

Dogs vs Cats

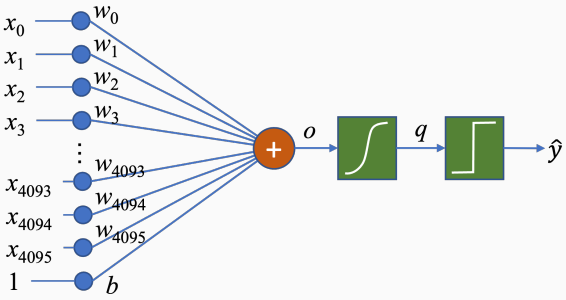
- 25000 images, resized to 64×64, grayscale
- Binary problem

In both cases the image is **flattened** into a vector of $64 \times 64 = 4096$ features.

Keep this in mind all session

Flattening throws away the fact that neighbouring pixels are related. Everything we do until session 3 ignores the structure of the image. The accuracy we get on Dogs vs Cats will remind us of that.

A single-layer network for binary classification



1. **Linear combination**

$$o = \boldsymbol{w}^{\mathsf{T}}\boldsymbol{x} + b$$

2. **Activation**, bounded range

$$q = g(o) = \frac{1}{1 + e^{-o}}$$

3. **Threshold**, the hard decision

$$\hat{y} = \mathbb{I}\left[q \geq \tfrac{1}{2}\right]$$

We cannot train on the accuracy

The accuracy is what we care about, but it is useless as a training criterion: the threshold is not differentiable, and its derivative is zero everywhere else.

We need a loss, a differentiable measure of disagreement between y and q:

- Square error: $\ell_2(y, q) = (y - q)^2$
- Absolute error: $\ell_1(y, q) = |y - q|$
- Cross entropy: $\ell_{\text{CE}}(y, q) = -y \log q - (1 - y) \log(1 - q)$

and we minimize the **empirical risk** $R(\boldsymbol{w}, b) = \sum_k \ell(y_k, q_k)$.

This is logistic regression

A network with probabilistic outputs defines a probability model:

$$P(y = 1 \mid \boldsymbol{w}, b, \boldsymbol{x}) = g(\boldsymbol{w}^\intercal \boldsymbol{x} + b)$$

$$\begin{aligned} \mathrm{NLL}(\boldsymbol{w}, b) &= -\sum_k \log P(y_k \mid \boldsymbol{w}, b, \boldsymbol{x}_k) \\ &= -\sum_k \big(y_k \log q_k + (1 - y_k) \log(1 - q_k)\big) = \sum_k \ell_{\mathrm{CE}}(y_k, q_k) \end{aligned}$$

The point

Minimizing the cross entropy *is* maximum likelihood. Everything you know about logistic regression carries over; what is new is that we are about to stack these units.

Training: gradient descent

Chain rule, one step at a time:

$$\frac{\partial \text{NLL}}{\partial \boldsymbol{w}} = \underbrace{\frac{\partial \text{NLL}}{\partial q}}_{\frac{1-y}{1-q} - \frac{y}{q}} \cdot \underbrace{\frac{\partial q}{\partial o}}_{q(1-q)} \cdot \underbrace{\frac{\partial o}{\partial \boldsymbol{w}}}_{\boldsymbol{x}} = \sum_k (q_k - y_k)\, \boldsymbol{x}_k$$

so the learning rules are remarkably simple:

$$\boldsymbol{w}_{n+1} = \boldsymbol{w}_n + \rho_n \sum_k (y_k - q_{k,n})\, \boldsymbol{x}_k$$

$$b_{n+1} = b_n + \rho_n \sum_k (y_k - q_{k,n})$$

error $\times$ input: this shape will come back in every layer of every network.

Four functions, and that is the whole network

```
def forward(w, b, x):
    return logistic(x @ w + b)

def loss(y, q):
    return -(y @ np.log(q)
             + (1-y) @ np.log(1-q)) / len(y)

def backward(y, q, x):
    w_grad = x.T @ (q - y) / len(y)
    b_grad = np.mean(q - y)
    return w_grad, b_grad

def accuracy(y, q):
    return np.mean(y == (q >= 0.5))
```

Anatomy of any network

- topology → `forward`
- criterion → `loss`
- gradients → `backward`
- update rule → the training loop

Only `backward` will become hard. In session 2 we will stop writing it.

Your turn

Notebook DL1, section 3.1 • 15 minutes

1. Run section 3.1 as it is, on the **binary digits** problem.
2. **Exercise 1.** Change the number of epochs and the learning rate ρ. Find a setting that diverges, and one that is clearly too slow.
3. **Exercise 2.** Repeat on **Dogs vs Cats**: `get_dataset('DogsCats')`. Get the best accuracy you can.

Be ready to answer: why is the performance so different between the two datasets?

Checkpoint

What you should be seeing

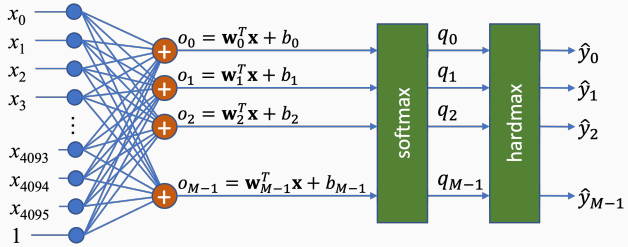
Dataset	validation accuracy
Digits 0 vs 1	≈ 0.95
Dogs vs Cats	≈ 0.60

Why the difference?

- The digits are centred, same pose, same lighting: a *linear* combination of pixels separates them.
- A dog is not in a fixed position. The same pixel means nothing on its own.
- A linear model on raw pixels cannot represent "has a snout somewhere".

This is the argument for hidden layers, and later for convolutions.

From 2 classes to M classes



One-hot encoding of the label:

$$\boldsymbol{y} = (0, \ldots, 1, \ldots, 0)^\intercal$$

One linear combination per class:

$$\boldsymbol{o} = \boldsymbol{W}\boldsymbol{x} + \boldsymbol{b}$$

Softmax activation:

$$q_i = \frac{e^{o_i}}{\sum_j e^{o_j}} \;\Rightarrow\; q_i \in [0, 1],\ \sum_i q_i = 1$$

Class prediction: `hardmax`, i.e. $\arg\max_i q_i$.

The gradient has the same shape

Cross entropy for M classes: $\ell_{\mathrm{CE}}(\boldsymbol{y}, \boldsymbol{q}) = -\sum_j y_j \log q_j$

Working through the softmax Jacobian, everything collapses:

$$\frac{\partial \ell}{\partial \boldsymbol{W}} = (\boldsymbol{q} - \boldsymbol{y})\, \boldsymbol{x}^{\mathsf{T}} \qquad \frac{\partial \ell}{\partial \boldsymbol{b}} = \boldsymbol{q} - \boldsymbol{y}$$

Same rule as before

error $\times$ **input**, again. The messy Jacobian of the softmax cancels exactly against the derivative of the cross entropy. This is not a coincidence: it is why this pair of (activation, loss) is the standard choice.

A detail that bites in practice

```
# overflows for large scores
def softmax(t):
    e = np.exp(t)
    return e / e.sum(axis=1, keepdims=True)
```

```
# the constant cancels out
def softmax(t):
    e = np.exp(t - t.max(axis=1,
                         keepdims=True))
    return e / e.sum(axis=1, keepdims=True)
```

$$\frac{e^{o_i}}{\sum_j e^{o_j}} = \frac{e^{o_i - c}}{\sum_j e^{o_j - c}} \quad \text{for any } c, \qquad \text{take } c = \max_j o_j$$

Half of the numerical problems in deep learning are of this kind. Libraries do this for you, which is one of the reasons to use them.

Your turn

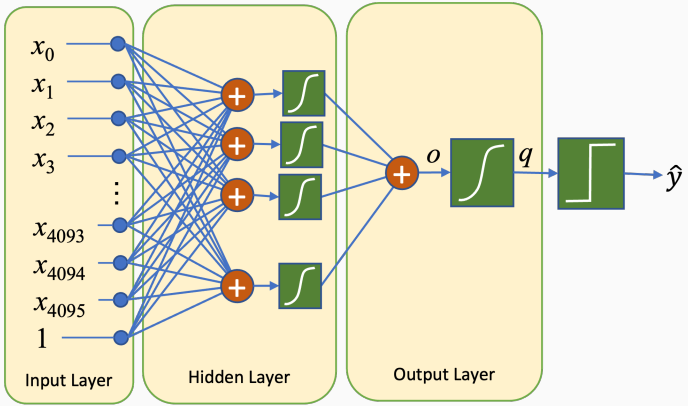
Notebook DL1, section 3.2 • 12 minutes

1. Run the 10-class network on the digits dataset.
2. **Exercise 3.** Explore epochs and learning rate.
3. **Exercise 4.** Compute the **confusion matrix** on the validation set. Which digits get confused with which?
4. **Exercise 5** (to discuss, no code): one 10-class network, or ten binary classifiers? What changes?

Multi-layer networks

So far every model is linear. We could apply a fixed non-linear transformation $\boldsymbol{z} = \boldsymbol{f}(\boldsymbol{x})$ first, as in non-linear regression. But a *fixed* transformation is a guess.

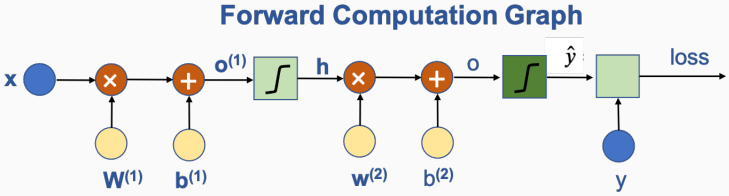
Better idea: parametrize the transformation itself and learn it too. That is a multi-layer perceptron.



Vocabulary and consequences

- Layers are counted ignoring the input layer; intermediate ones are **hidden layers**
- **Non-linear** activations are what makes the whole thing non-linear: stacking linear layers gives one linear layer
- This is already **deep learning**. Nothing else is needed, as long as the derivatives exist
- How many layers, how many neurons? Exploration, guided by validation
- More data $\rightarrow$ more parameters you can afford

A 2-layer network



- Hidden layer: n_h units, tanh activation
- Output layer: one unit, logistic
- Loss: cross entropy

$$\boldsymbol{h} = \tanh\left(\boldsymbol{W}^{(1)}\boldsymbol{x} + \boldsymbol{b}^{(1)}\right)$$
$$q = \text{logistic}\left(\boldsymbol{w}^{(2)\mathsf{T}}\boldsymbol{h} + b^{(2)}\right)$$

Back-propagation is the chain rule, read backwards

Output layer: identical to before, with $\boldsymbol{h}$ in the place of $\boldsymbol{x}$

$$\frac{\partial \ell}{\partial \boldsymbol{w}^{(2)}} = (q - y)\, \boldsymbol{h}$$

Hidden layer: one more link in the chain

$$\frac{\partial \ell}{\partial \boldsymbol{W}^{(1)}} = \underbrace{(q - y)}_{\text{output error}} \Big[\underbrace{\boldsymbol{w}^{(2)}}_{\text{how much each unit contributed}} \odot \underbrace{(\mathbf{1} - \boldsymbol{h}^2)}_{\tanh'} \Big] \boldsymbol{x}^\mathsf{T}$$

Why "backwards"

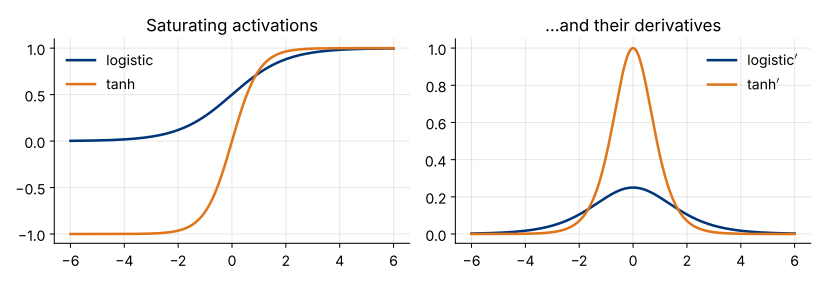
The error computed at the output is *propagated back*, multiplied at each step by the weights of the layer and by the derivative of its activation. Each layer reuses the quantity computed by the layer above it.

Your turn

Notebook DL1, section 3.3 • 12 minutes

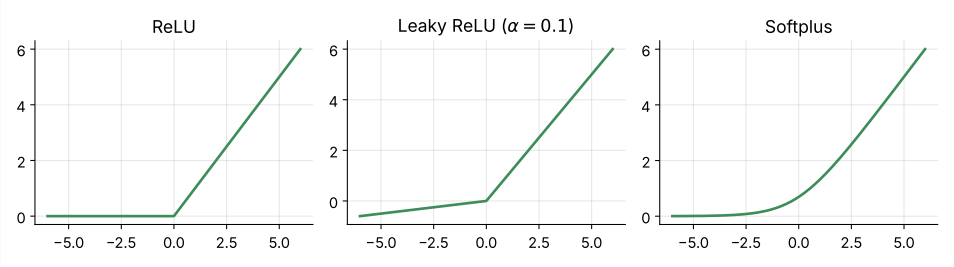
1. Run the gradient check. Then break it on purpose: change `(1 - h**2)` back to `(1 - h)**2` and look at what the check reports, and at what the loss curve reports.
2. Train the 2-layer network on both datasets.
3. **Exercise 6.** Vary n_h, ρ and the number of epochs. Find a divergent setting and explain it in terms of saturation.
4. **Exercise 7** (if you have time): solve the same problems with `sklearn`'s `MLPClassifier`.

The problem with saturating activations



The derivative vanishes away from the origin. In a deep network these factors multiply: with a few layers, the gradient reaching the first layer is numerically zero. This is the **vanishing gradient** problem.

Activation functions used in practice



- **ReLU**: $\max(0, t)$. Derivative is a step. The default choice for hidden layers
- **Leaky ReLU / PReLU**: $\max(\alpha t, t)$, avoids exactly-zero derivatives
- **Softplus**: $\log(1 + e^t)$, a smooth ReLU; its derivative is the logistic

Activations must be *non-linear*; they need not be bounded or probabilistic — except, usually, at the output layer.

And for regression?

Two changes, nothing else:

- **Linear** activation at the output unit (the target is not in $[0, 1]$)
- **Square loss** instead of the cross entropy: $\ell(y, \hat{y}) = (y - \hat{y})^2$

Same machinery

Topology, loss, forward, backward, update. Changing the task changes the last layer and the loss; the rest of the network, and all of back-propagation, is untouched.

Session 1 wrap-up

What we built

- A single neuron = logistic regression
- Softmax output for M classes
- A hidden layer, and back-propagation
- Activation functions and why they matter

What does not scale

- Deriving gradients by hand for each topology
- A wrong gradient is hard to detect
- Nothing here runs on a GPU

Before the next session

Work through the two short official tutorials linked at the end of the notebook: PyTorch **tensors** and **autograd**. 30 minutes, and session 2 will be much easier.

Session 2 • notebook DL2

Deep networks with PyTorch

What PyTorch gives us

- **Tensors**, numpy-like, that can live on a GPU
- **Automatic differentiation**: no more `backward` by hand
- **Pre-defined components**: layers, losses, optimizers, data loaders
- **Several levels of abstraction**, so you can drop down when you need to

Today's plan

We rewrite *the same model* four times, climbing one level of abstraction each time.
The point is not the final version: it is understanding what each level hides.

Tensors

```
import torch

x = torch.rand((100, 200))
t = torch.from_numpy(my_array)

x.size()           # shape
x.t()              # transpose
x.mm(x.t())        # matrix product
x.add(y)           # elementwise
x.add_(y)          # ... in place

x = x.to(device)  # move to the GPU
```

Two traps

- `from_numpy` **shares memory** with the array
- torch layers use `float32`; numpy defaults to `float64`. Mixing them is a very common error

Automatic differentiation

```
x.requires_grad = True

y = (3 * torch.log(x)).sum()
y.backward()

x.grad          # == 3/x
```

PyTorch records every operation on tensors with `requires_grad` into a **computation graph**. Calling `backward()` walks it backwards, applying the chain rule — exactly what we did by hand in session 1.

Gradients accumulate

A second `backward()` adds to `.grad` instead of replacing it. This is deliberate (it is useful), and it is why every training loop calls `zero_grad()`.

Your turn

Notebook DL2, sections 1.2–1.3 • 10 minutes

1. Run the tensor and autograd cells.
2. **Exercise 1**, the five steps: build a tensor from the data, take its square root, sum the squares, differentiate, and check the result by hand.
3. Step 1.5 is the important one: make `backward()` runnable twice with the same result. *Why does it fail the second time?*

Level 1 — tensors and autograd only

```
W = .1 * torch.randn(n_in, n_out)
W.requires_grad_()
b = torch.zeros(n_out, requires_grad=True)

for epoch in range(epochs):
    pred = model(W, b, X_train)
    loss = nll(y_train, pred)
    loss.backward()

    with torch.no_grad():
        W -= rho * W.grad
        b -= rho * b.grad
        W.grad.zero_()
        b.grad.zero_()
```

Compared with session 1, exactly one thing disappeared: the `backward` function.

`torch.no_grad()` tells PyTorch not to record the weight update itself as part of the graph.

Levels 2 and 3 — nn.Module, nn.Linear, losses

```
class MyNet(nn.Module):
    def __init__(self, nin, nout):
        super().__init__()
        self.W = nn.Parameter(.1*torch.randn(nin,
     nout))
        self.b = nn.Parameter(torch.zeros(nout))

    def forward(self, x):
        return softmax(x.mm(self.W) + self.b)
```

```
class MyNet(nn.Module):
    def __init__(self, nin, nout):
        super().__init__()
        self.lin = nn.Linear(nin, nout)

    def forward(self, x):
        return self.lin(x)

loss_func = nn.CrossEntropyLoss()
```

Level 2. `nn.Parameter` registers a tensor as trainable; the loop becomes `for p in net.parameters()`.

Level 3. `nn.Linear` brings its own weights *and* a sensible initialization. `nn.CrossEntropyLoss` includes the softmax, so the network outputs raw scores and takes integer labels, not one-hot.

The faster convergence you will see is due to the initialization, not to any change in the model.

Level 4 — optimizers

```
from torch import optim

opt = optim.SGD(net.parameters(), lr=0.1)

for epoch in range(epochs):
    pred = net(X_train)
    loss = loss_func(pred, y_train)
    loss.backward()
    opt.step()
    opt.zero_grad()
```

The update rule is now a replaceable object: `SGD`, `Adam`, `RMSprop`, with momentum, schedulers, and so on.

`opt.step()` applies the rule to every registered parameter; `opt.zero_grad()` clears the accumulated gradients.

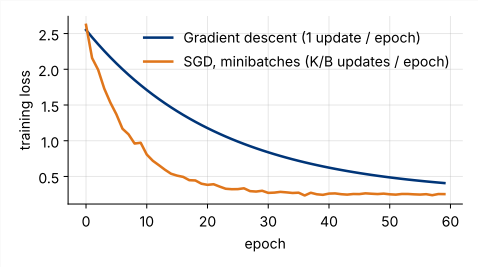
Four levels, same model. From here on we only use the last one.

Your turn

Notebook DL2, sections 2.1–2.3 • 14 minutes

1. Run the four versions and check they give the same thing.
2. **Exercise 2.** Replace SGD by **Adam**, and add a learning rate **scheduler** (`ExponentialLR`).
3. Compare the curves. How many epochs does Adam need to reach what SGD reached?

Minibatches: from GD to SGD



Until now, one gradient over the *whole* training set per epoch: one update per epoch.

SGD with minibatches: split the epoch into batches of B samples and update after each one. With K samples that is K/B updates per epoch, each one noisier but almost as useful.

```
train_ds = TensorDataset(X_train, y_train)
train_dl = DataLoader(train_ds,
                      batch_size=64,
                      shuffle=True)

for xb, yb in train_dl:
    ...
```

Multi-layer networks in five lines

```
my_net = nn.Sequential(
    nn.Linear(n_in, 200), nn.ReLU(),
    nn.Linear(200, 50),   nn.ReLU(),
    nn.Linear(50, 20),    nn.ReLU(),
    nn.Linear(20, n_out))
```

Compare with session 1: we derived the gradients of a *two*-layer network over two slides. This network has four, and we need not to differentiate anything.

Notebook DL2, sections 2.4–2.5 • 9 minutes

Run the `DataLoader` version and the `nn.Sequential` network. Look at the final accuracy, and then look at the **gap between the training and the validation curves**. We will come back to that gap in a minute.

Over-fitting, read from the curves

What you should have seen

- training accuracy $\rightarrow 100\,\%$
- validation accuracy saturates around $79\,\%$
- the validation *loss* stops going down and starts to grow

The two symptoms are different

Accuracy saturating = the model has learnt what it can.
Validation loss *growing* = the model is becoming confidently wrong.

Cross-validation over every possible topology is hopeless: too many configurations. In practice we combine it with techniques that *limit* the effective capacity.

Penalty terms: L2 and L1

$$R(\boldsymbol{w}) + \lambda \|\boldsymbol{w}\|^2 \qquad\qquad R(\boldsymbol{w}) + \lambda \|\boldsymbol{w}\|_1$$

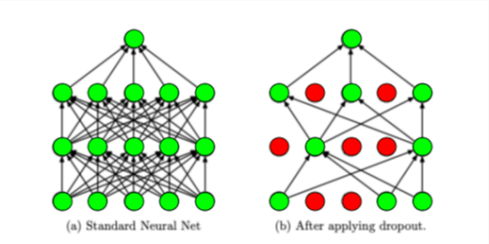
```
opt = optim.SGD(net.parameters(), lr=0.01, weight_decay=1e-2)
```

What to expect on our dataset

Accuracy barely moves ($0.792 \rightarrow 0.792$), but the validation loss at the end of training drops from 1.08 to 0.79.

The regularized network does not make fewer mistakes: it makes them *less confidently*. Which of the two numbers matters is a decision about your problem, not about the method.

Dropout




```
nn.Sequential(
    nn.Linear(n_in, 200), nn.ReLU(), nn.Dropout(0.5),
    nn.Linear(200, 50),   nn.ReLU(), nn.Dropout(0.5),
    nn.Linear(50, 20),    nn.ReLU(),
    nn.Linear(20, n_out))
```

A fraction p of the units is switched off at random **at each training step**. No unit can rely on any other being present.

At *inference* time dropout is off and all units are used.

The classic bug

`net.train()` and `net.eval()` switch that behaviour. Evaluating without `eval()` gives you random predictions.

Early stopping, done properly

- Monitor the validation metric at every epoch
- Keep a copy of the best model, not the last one
- Do not stop at the first bad epoch: wait **patience** epochs without improvement

On our data

Monitoring the **loss** selects epoch $\approx$ 139, with accuracy 0.760.
The best **accuracy**, 0.792, happens much later.

The criterion you monitor is the model you get. Choose it on purpose.

Your turn

Notebook DL2, section 3 • 8 minutes in class, finish at home

1. Run the three experiments: weight decay, dropout, early stopping.
2. **Exercise 3.** Sweep `weight_decay` from 10^{-5} to 10^{-1}.
3. **Exercise 4.** Sweep the dropout probability. Note that the *training* accuracy gets worse — and that this is not a failure.
4. **Exercise 5.** Combine the three. Is the combination better than the best one alone?
5. Switch early stopping to `monitor='accuracy'` and see which model it picks.

Session 2 wrap-up

The ladder

1. tensors + autograd
2. `nn.Module` / `nn.Parameter`
3. `nn.Linear` / `nn.CrossEntropyLoss`
4. `torch.optim`, `DataLoader`, `nn.Sequential`

Controlling over-fitting

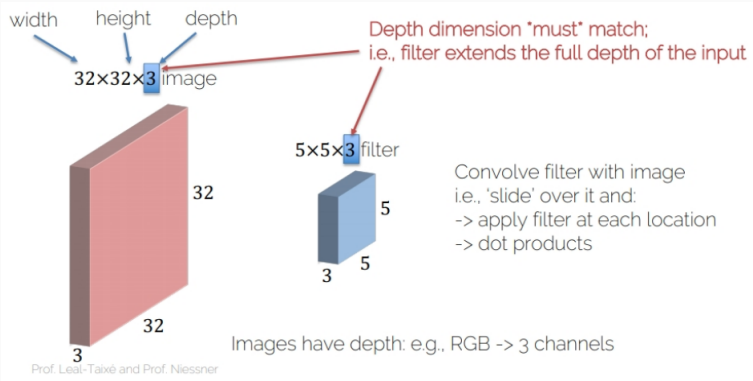
- weight decay (L2)
- dropout
- early stopping
- data augmentation → next session

Still missing: every model so far sees an image as 4096 unrelated numbers.

Session 3 • notebook DL3

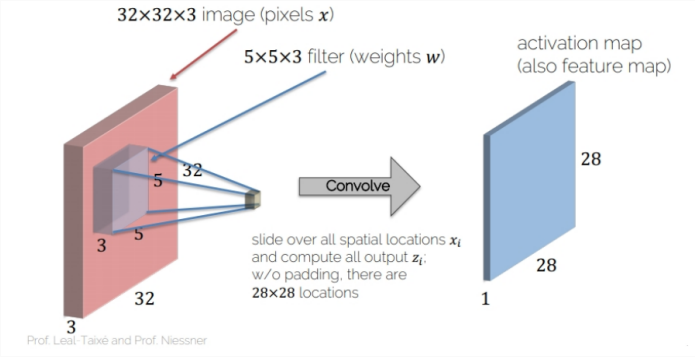
Convolutional networks

The convolution operator



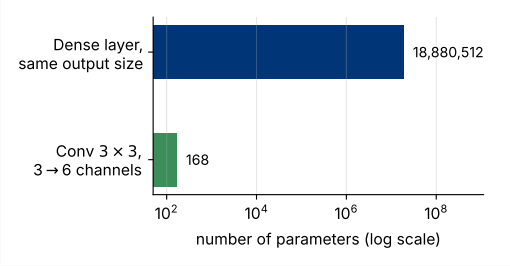
A small **kernel** slides over the input; at each position we compute a weighted sum. The kernel weights are the parameters, and they are the same at every position.

Activation maps



Each filter produces one **activation map** (a channel). A layer with F filters turns a $C \times H \times W$ input into an $F \times H' \times W'$ output.

Why a dense layer is the wrong tool for an image



- Do we really need every output to depend on every pixel?
- A dense layer has an enormous number of parameters: expensive, and prone to over-fitting
- And it is not *translation invariant*: an edge in the top-left corner and the same edge in the centre are learnt separately

The two ideas of a CNN

Local connectivity: each unit sees a small window.

Weight sharing: the same window is applied everywhere.

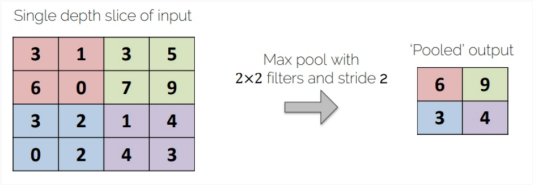
Number of parameters of the convolutional layer

```
conv = nn.Conv2d(in_channels=3, out_channels=6, kernel_size=3,
                 stride=1, padding=1)
```

$$H' = \left\lfloor \frac{H + 2\,\text{padding} - \text{kernel}}{\text{stride}} \right\rfloor + 1$$

- `padding=1` with `kernel=3` and `stride=1` $\Rightarrow$ the spatial size is preserved
- `stride=2` halves it
- the number of channels is `out_channels`, and has nothing to do with the spatial size
- parameters: out $\times$ in $\times$ k $\times$ k + out, independent of the image size

Pooling



Pooling **downsamples** the activation maps, combining a window of inputs into a single output:

- `nn.MaxPool2d(2)`: the maximum
- `nn.AvgPool2d(2)`: the average

It reduces the spatial size (and the computation that follows), and it gives a small amount of invariance to translation.

It has **no parameters**.

Your turn

Notebook DL3, section 2 • 12 minutes

1. Load CIFAR10 and look at the images. Use a GPU runtime (Runtime → Change runtime type).
2. Apply the convolution and the two pooling operators, and check the shapes.
3. Compare the parameter count of the convolution with that of an equivalent dense layer.
4. **Exercise 1.** Predict, *before running*, the output size with `padding=0`, with `stride=2`, and with `kernel_size=5`.

A complete CNN

```
class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 5, 3, padding=1)
        self.conv2 = nn.Conv2d(5, 10, 3, padding=1)
        self.relu = nn.ReLU(inplace=True)
        self.pool = nn.MaxPool2d(2, 2)
        self.fc = nn.Linear(10*8*8, 10)

    def forward(self, x):
        x = self.pool(self.relu(self.conv1(x)))
        x = self.pool(self.relu(self.conv2(x)))
        x = torch.flatten(x, 1)
        return self.fc(x)
```

$3 \times 32 \times 32 \to 5 \times 32 \times 32 \to 5 \times 16 \times 16$
$\to 10 \times 16 \times 16 \to 10 \times 8 \times 8 \to 640 \to 10$

Convolutional part extracts features, **dense part** classifies them. This is the standard shape of an image classifier.

The training loop is *exactly* the one from session 2.

Your turn

Notebook DL3, section 3 • 15 minutes

1. Train the CNN for 10 epochs and look at the curves and at the **per-class** accuracy. Which classes are hard? Does that make sense?
2. **Exercise 2.** Make the network bigger: more filters (16 and 32), a third convolutional block, a hidden dense layer. Which change buys the most accuracy per added parameter?

While it trains: compare the number of parameters of this CNN with a dense network on the same input.

Data augmentation

```
transform_augmented = transforms.Compose([
    transforms.RandomHorizontalFlip(),
    transforms.RandomCrop(32, padding=4),
    transforms.ToTensor(),
    transforms.Normalize(mean, std)])
```

Random transformations that **do not change the label**: a mirrored cat is a cat.

Applied *on the fly*, differently at each epoch, and only to the **training** set.

Which transformations are valid is a modelling decision

A vertical flip is fine for satellite images and wrong for street photos. For handwritten digits, a horizontal flip turns a 2 into something that is not a 2 — and rotating a 6 far enough gives a 9.

Transfer learning

1. Take a network trained on a much larger dataset (ResNet18 on ImageNet)
2. **Freeze** the convolutional backbone: it already knows how to extract general image features
3. Replace the last layer with one that has our number of classes
4. Train *only* that layer

The numbers

ResNet18: ≈ 11.2M parameters.
Trainable after freezing: 5130, i.e. 0.05 %.

And it beats, on a tenth of the data, a CNN trained from scratch on all of it.

On any image problem with a small dataset, this is the first thing to try.

Transfer learning in code

```
self.resnet = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)

for p in self.resnet.parameters():      # freeze the ResNet weights
    p.requires_grad = False

self.resnet.fc = nn.Linear(self.resnet.fc.in_features, 10)   # new head
```

- The pretrained network expects **its** input size and normalization (224×224, ImageNet statistics)
- The new layer is created with `requires_grad=True`, so it is the only one that trains
- **Fine-tuning**: unfreeze the backbone afterwards and continue with a much smaller learning rate

Your turn

Notebook DL3, sections 4–5 • 15 minutes

1. Train the same CNN with **data augmentation** and compare. **Exercise 3**: try more epochs, and other transformations.
2. Run the **transfer learning** section. Compare the accuracy, and the number of trainable parameters, with the CNN trained from scratch.
3. **Exercise 4.** Unfreeze the backbone and fine-tune with a small learning rate. Does it help? What does it cost?

Where this block leaves you

- A neural network is a **topology**, a **loss** and an **optimization rule**; back-propagation is the chain rule applied backwards
- A library removes the gradients from your hands and gives you optimizers, data loaders and regularization layers — but the bugs move to `train()/eval()`, to dtypes and to shapes
- A **convolutional** topology encodes locality and weight sharing, with orders of magnitude fewer parameters
- Most practical image problems start from a **pretrained** network

Next

The same ideas, applied to text: embeddings, and the topologies built on top of them.